

Strings — Methods & Template Literals

1. String Methods কী?

String-এর উপর বিভিন্ন ধরনের কাজ করার জন্য JavaScript অনেক built-in method দেয়।

যেমন:

```
const message = "Hello World";

console.log(message.includes("World"));
console.log(message.toUpperCase());
console.log(message.slice(0, 5));
```

String methods সাধারণত **original string** পরিবর্তন করে না, কারণ JavaScript strings immutable।

তারা সাধারণত **নতুন value return** করে।

2. Important String Methods

এই Doc-এ আমরা শিখব:

Method	কাজ
<code>indexOf()</code>	কোনো text-এর position খুঁজে বের করা
<code>includes()</code>	কোনো text আছে কিনা check করা
<code>slice()</code>	string-এর একটি অংশ বের করা
<code>split()</code>	string → array
<code>join()</code>	array → string
<code>trim()</code>	শুরু/শেষের whitespace remove করা
<code>replace()</code>	text replace করা
<code>replaceAll()</code>	সব matching text replace করা

এর সাথে:

- Method chaining

- Template literals
 - String interpolation
 - Multiline strings
 - Tagged templates
 - `String.raw`
-

3. `indexOf()`

`indexOf()` কোনো substring বা character string-এর কোথায় প্রথম পাওয়া গেছে সেটা return করে।

Syntax:

```
string.indexOf(searchValue);
```

Example:

```
const text = "Hello World";

console.log(text.indexOf("World"));
```

Output:

```
6
```

কারণ:

```
H e l l o   W o r l d
0 1 2 3 4 5 6 7 8 9 10
          ↑
        World
```

4. Character Search with `indexOf()`

```
const text = "JavaScript";

console.log(text.indexOf("J")); // 0
console.log(text.indexOf("a")); // 1
console.log(text.indexOf("S")); // 4
```

5. `indexOf()` যদি কিছু না পায়

যদি search value না পাওয়া যায়:

```
const text = "Hello";

console.log(text.indexOf("x"));
```

Output:

```
-1
```

এটা খুব important!

```
found → index >= 0
not found → -1
```

6. `indexOf()` Case Sensitive

```
const text = "JavaScript";

console.log(text.indexOf("java"));
```

Output:

```
-1
```

কারণ:

```
JavaScript
^
J uppercase
```

আর:

```
console.log(text.indexOf("Java"));
```

Output:

```
0
```

7. `indexOf()` with Second Argument

তুমি চাইলে কোন index থেকে search শুরু হবে সেটাও দিতে পারো।

Syntax:

```
string.indexOf(searchValue, startIndex);
```

Example:

```
const text = "banana";

console.log(text.indexOf("a"));      // 1
console.log(text.indexOf("a", 2));  // 3
```

এখানে দ্বিতীয় search index `2` থেকে শুরু হয়েছে।

8. `includes()`

`includes()` check করে string-এর মধ্যে নির্দিষ্ট text আছে কিনা।

Return করবে:

```
true
```

অথবা:

```
false
```

Example:

```
const message = "Hello World";

console.log(message.includes("World"));
```

Output:

```
true
```

9. `includes()` VS `indexOf()`

দুটো দিয়েই search করা যায়, কিন্তু উদ্দেশ্য আলাদা।

```
const text = "Hello World";

console.log(text.indexOf("World"));
console.log(text.includes("World"));
```

Output:

```
6
true
```

Mental Model

```
indexOf()
→ "কোথায় আছে?"

includes()
→ "আছে কি?"
```

যদি শুধু existence check করতে চাও:

```
text.includes("World");
```

বেশি readable।

10. `includes()` is Case Sensitive

```
const text = "JavaScript";

console.log(text.includes("javascript"));
```

Output:

```
false
```

কারণ uppercase/lowercase আলাদা।

Case-insensitive check করতে চাইলে:

```
const text = "JavaScript";

console.log(text.toLowerCase().includes("javascript"));
```

Output:

```
true
```

11. `slice()`

`slice()` string-এর একটি অংশ বের করে নতুন **string** return করে।

Syntax:

```
string.slice(start, end);
```

`end` indexটি include হয় না।

Example:

```
const text = "JavaScript";

console.log(text.slice(0, 4));
```

Output:

```
Java
```

কারণ:

```
J a v a S c r i p t
0 1 2 3 4
↑      ↑
start  end
```

4 include হয়নি।

12. `slice()` — Start Only

শুধু start দিলে start থেকে শেষ পর্যন্ত নেয়।

```
const text = "JavaScript";

console.log(text.slice(4));
```

Output:

```
Script
```

13. `slice()` with Negative Index

এখানে `slice()` খুব powerful।

```
const text = "JavaScript";

console.log(text.slice(-6));
```

Output:

```
Script
```

Negative index শেষ দিক থেকে count করে।

```
JavaScript
  Script
    ↑
  -6
```

14. Negative Start and End

```
const text = "JavaScript";

console.log(text.slice(-6, -1));
```

Output:

```
Scrip
```

কারণ end `-1` characterটি include হয় না।

15. `slice()` Does Not Mutate Original

```
const text = "JavaScript";

const result = text.slice(0, 4);

console.log(text);
console.log(result);
```

Output:

```
JavaScript
Java
```

Original string unchanged

16. `split()`

`split()` একটি string-কে separator অনুযায়ী **array-তে convert করে**।

Syntax:

```
string.split(separator);
```

Example:

```
const fruits = "apple,banana,mango";

console.log(fruits.split(","));
```

Output:

```
["apple", "banana", "mango"]
```

Mental Model

```
String
  ↓ split()
Array
```

17. Split by Space

```
const sentence = "I love JavaScript";

console.log(sentence.split(" "));
```

Output:

```
["I", "love", "JavaScript"]
```

18. Split into Individual Characters

Empty string separator দিলে:

```
const word = "Hello";

console.log(word.split(""));
```

Output:

```
["H", "e", "l", "l", "o"]
```

তবে Unicode/emoji-এর ক্ষেত্রে `split("")` সবসময় user-perceived character অনুযায়ী কাজ করে না।

19. `split()` with Limit

তুমি চাইলে কতগুলো element পর্যন্ত তৈরি হবে সেটা limit করতে পারো।

```
const text = "A-B-C-D";

console.log(text.split("-", 2));
```

Output:

```
["A", "B"]
```

20. `join()`

`join()` array-এর elements একসাথে করে string বানায়।

Syntax:

```
array.join(separator);
```

Example:

```
const fruits = ["apple", "banana", "mango"];

console.log(fruits.join(", "));
```

Output:

```
apple, banana, mango
```

Mental Model

```
Array
  ↓ join()
String
```

21. `split()` + `join()`

দুটো একসাথে খুব useful।

```
const text = "hello-world-javascript";

const result = text.split("-").join(" ");

console.log(result);
```

Output:

```
hello world javascript
```

22. Reverse String with `split()`, `reverse()`, `join()`

এটা classic JavaScript exercise।

```
const word = "hello";

const reversed = word.split("").reverse().join("");

console.log(reversed);
```

Output:

```
olleh
```

Flow:

```
"hello"
  ↓ split("")
["h", "e", "l", "l", "o"]
  ↓ reverse()
["o", "l", "l", "e", "h"]
  ↓ join("")
"olleh"
```

23. `trim()`

`trim()` string-এর beginning এবং ending-এর whitespace remove করে।

```
const username = "  Shariar  ";

console.log(username.trim());
```

Output:

```
Shariar
```

24. `trim()` Only Removes Outer Whitespace

```
const text = "  Hello World  ";

console.log(text.trim());
```

Output:

```
Hello World
```

মাত্রের whitespace remove হবে না।

```
"Hello World"
    ↑
  space
```

এটা থাকবে।

25. `trimStart()` and `trimEnd()`

শুধু beginning থেকে remove:

```
const text = "  Hello  ";

console.log(text.trimStart());
```

Output:

```
Hello
```

শুধু ending থেকে:

```
console.log(text.trimEnd());
```

Output:

```
Hello
```

26. Practical Use of `trim()`

Form input-এর ক্ষেত্রে খুব common।

```
const username = input.value.trim();

if (username === "") {
  console.log("Username is required");
}
```

এতে user শুধু spaces দিলে সেটাকেও effectively empty input হিসেবে handle করা যায়।

27. `replace()`

`replace()` string-এর একটি অংশ replace করে নতুন string return করে।

```
const text = "I love JavaScript";

const result = text.replace("JavaScript", "React");

console.log(result);
```

Output:

```
I love React
```

Original unchanged:

```
console.log(text);
```

Output:

```
I love JavaScript
```

28. `replace()` Only Replaces First Match

এটা খুব important!

```
const text = "apple apple apple";

console.log(text.replace("apple", "mango"));
```

Output:

```
mango apple apple
```

শুধু first matching occurrence replace হয়েছে।

29. `replaceAll()`

সব matching occurrence replace করতে:

```
const text = "apple apple apple";

console.log(text.replaceAll("apple", "mango"));
```

Output:

```
mango mango mango
```

30. `replace()` with Regular Expression

Regex ব্যবহার করেও সব occurrence replace করা যায়।

```
const text = "apple apple apple";

const result = text.replace(/apple/g, "mango");

console.log(result);
```

Output:

```
mango mango mango
```

Beginner level-এ simple text replacement-এর জন্য `replaceAll()` অনেক readable।

31. `replace()` Uses Callback Too

`replace()` -এ replacement হিসেবে function-ও দেওয়া যায়।

```
const text = "hello world";

const result = text.replace("world", (match) => {
  return match.toUpperCase();
});

console.log(result);
```

Output:

```
hello WORLD
```

এটা পরে advanced string manipulation এবং regex শেখার সময় আরও useful হবে।

32. Method Chaining

String methods একটার পর একটা chain করা যায়।

Example:

```
const username = "  Shariar  ";

const result = username.trim().toUpperCase();

console.log(result);
```

Output:

```
SHARIAR
```

Flow:

```
"  Shariar  "
    ↓ trim()
"Shariar"
    ↓ toUpperCase()
"SHARIAR"
```

33. More Method Chaining

```
const text = "  hello javascript  ";

const result = text
  .trim()
  .toUpperCase()
  .replace("JAVASCRIPT", "REACT");

console.log(result);
```

Output:

```
HELLO REACT
```

Method chaining readable code লিখতে সাহায্য করে, তবে chain খুব বড় হলে intermediate variables ব্যবহার করা ভালো।

34. Template Literals

Template literals string লেখার একটি modern syntax।

Backtick ব্যবহার করতে হয়:

```
const message = `Hello World`;
```

এখানে:

```
`
```

এটাকে backtick বলা হয়।

35. Why Template Literals?

Template literals-এর সবচেয়ে বড় সুবিধা হলো **string interpolation**।

ধরো:

```
const name = "Shariar";  
const age = 31;
```

Traditional way:

```
const message = "My name is " + name + " and I am " + age + " years old.";
```

Template literal:

```
const message = `My name is ${name} and I am ${age} years old.`;
```

এটা অনেক cleaner।

36. `` ${ } `` — String Interpolation

Template literal-এর মধ্যে JavaScript expression লিখতে:

```
${expression}
```

ব্যবহার করি।

Example:

```
const name = "Shariar";  
  
console.log(`Hello ${name}`);
```

Output:

```
Hello Shariar
```

37. Expression Inside Template Literal

শুধু variable নয়, expression-ও দেওয়া যায়।

```
const a = 10;
const b = 20;

console.log(`Total: ${a + b}`);
```

Output:

```
Total: 30
```

আর:

```
const age = 31;

console.log(`Next year I will be ${age + 1}`);
```

Output:

```
Next year I will be 32
```

38. Function Call Inside `${ }`

```
const name = "shariar";

console.log(`Name: ${name.toUpperCase()}`);
```

Output:

```
Name: SHARIAR
```

39. Conditional Expression

Template literal-এর মধ্যে expression হিসেবে ternary operator-ও ব্যবহার করা যায়।

```
const age = 20;

console.log(`Status: ${age >= 18 ? "Adult" : "Minor"}`);
```

Output:

```
Status: Adult
```

40. Multiline Strings

Template literals-এর আরেকটি বড় সুবিধা হলো multiline string।

```
const message = `
  Hello Shariar,
  Welcome to JavaScript.
  Keep learning!
`;

console.log(message);
```

Traditional quoted strings-এ multiline handling-এর জন্য escape বা concatenation লাগতে পারে, কিন্তু template literals সরাসরি line breaks preserve করতে পারে।

41. Practical React Example

React UI-তে dynamic text:

```
const username = "Shariar";
const notificationCount = 5;

return (
  <h2>
    {`Hello ${username}, you have ${notificationCount} notifications.`}
  </h2>
);
```

তবে JSX-এর ভিতরে সাধারণত সরাসরি:

Hello {username}, you have {notificationCount} notifications.

এটাই বেশি natural।

42. Template Literal vs Normal String

Feature	<code>' '</code> / <code>" "</code>	<code>` `</code>
Normal string	✓	✓
Variable interpolation	✗	✓
<code>\${}</code>	✗	✓
Multiline	Limited	✓
Expressions	✗	✓

43. Tagged Templates

Tagged template literals একটু advanced feature।

একটি function-কে template literal-এর আগে বসানো যায়:

```
tag`Hello ${name}`;
```

এখানে:

```
tag → function  
`Hello ${name}` → template literal
```

JavaScript এই template-কে function-এর কাছে structured arguments হিসেবে পাঠায়।

44. Basic Tagged Template Example

```
function tag(strings, name) {  
  console.log(strings);  
  console.log(name);  
}  
  
const userName = "Shariar";  
  
tag`Hello ${userName}`;
```

এখানে function-এর প্রথম argument হিসেবে static string অংশগুলো আসে এবং interpolation-এর values আলাদা argument হিসেবে আসে।

Conceptually:

```
Template  
  ↓  
tag function  
  ↓  
strings + interpolated values
```

45. Understanding **strings** in Tagged Templates

```
function tag(strings, name) {  
  console.log(strings);  
  console.log(name);  
}  
  
const userName = "Shariar";  
  
tag`Hello ${userName}!`;
```

Conceptually:

```
strings → ["Hello ", "!"]  
name    → "Shariar"
```

অর্থাৎ template literal-এর static parts এবং dynamic values আলাদা করে পাওয়া যায়।

46. Multiple Expressions in Tagged Templates

```
function tag(strings, name, age) {  
  console.log(strings);  
  console.log(name);  
  console.log(age);  
}  
  
const name = "Shariar";  
const age = 31;  
  
tag`Name: ${name}, Age: ${age}`;
```

Conceptually:

```
strings → ["Name: ", ", ", "Age: ", ""]  
name    → "Shariar"  
age     → 31
```

47. Why Tagged Templates Exist?

Tagged templates সাধারণ everyday string concatenation-এর জন্য নয়।

এগুলো useful হতে পারে:

- Custom formatting
- Localization
- Sanitization
- SQL-like template systems
- CSS-in-JS libraries
- Domain-specific languages
- Advanced string processing

React ecosystem-এর কিছু libraries এই concept ব্যবহার করে।

48. `String.raw`

`String.raw` একটি special tagged template function যা raw string representation পেতে সাহায্য করে।
বিশেষ করে escape sequences-এর behavior বুঝতে এটি useful।

Example:

```
const text = String.raw`Hello\nWorld`;  
  
console.log(text);
```

এখানে `\n` actual newline হিসেবে interpret না হয়ে raw backslash + `n` sequence হিসেবে রাখা হয়।

Output conceptually:

```
Hello\nWorld
```

49. Normal Template Literal vs `String.raw`

Normal:

```
const text = `Hello\nWorld`;  
  
console.log(text);
```

Output:

```
Hello  
World
```

`String.raw`:

```
const text = String.raw`Hello\nWorld`;  
  
console.log(text);
```

Output:

```
Hello\nWorld
```

Mental Model

```
Normal template literal  
→ escape sequences interpreted  
  
String.raw  
→ raw representation
```

50. Practical Use of `String.raw`

Windows-style path-এর মতো strings handle করার সময় conceptually useful হতে পারে:

```
const path = String.raw`C:\Users\Shariar\Documents`;  
  
console.log(path);
```

এখানে backslashes raw form-এ রাখা যায়।

51. Important: `String.raw` is Not "No Escaping Everywhere"

`String.raw` মূলত tagged-template mechanism-এর মাধ্যমে raw template text access করার সুবিধা দেয়। এটা string-এর প্রতিটি ধরনের escaping বা Unicode processing permanently বন্ধ করে দেয়—এভাবে ভাবা ঠিক নয়।

Beginner level-এ মনে রাখো:

`String.raw` → *template literal-এর raw backslash representation নিয়ে কাজ করার জন্য useful!*

52. `split()` + `join()` + `replace()` — Real Example

ধরো:

```
const title = "learn-javascript-fast";
```

আমরা এটাকে:

```
Learn Javascript Fast
```

করতে চাই।

একটি simple approach:

```
const result = title
  .split("-")
  .join(" ");

console.log(result);
```

Output:

```
learn javascript fast
```

তারপর capitalization:

```
const result = title
  .split("-")
  .map(word => word[0].toUpperCase() + word.slice(1))
  .join(" ");

console.log(result);
```

Output:

```
Learn Javascript Fast
```

এখানে string → array → transformed array → string হয়েছে।

53. Real Example — Clean User Input

```
const input = "  Hello World  ";

const cleanInput = input
  .trim()
  .toLowerCase();

console.log(cleanInput);
```

Output:

```
hello world
```

Form/API data processing-এ এই pattern খুব common।

54. Real Example — Check Email Domain

```
const email = "shariar@gmail.com";

if (email.includes("@gmail.com")) {
  console.log("Gmail address");
}
```

আরও robust validation-এর জন্য শুধু `includes()` যথেষ্ট নয়—real email validation-এর আলাদা considerations আছে।

55. Real Example — Mask Sensitive Text

```
const phone = "01712345678";

const masked = phone.slice(0, 3) + "****" + phone.slice(-4);

console.log(masked);
```

Output:

```
017****5678
```

এখানে:

```
slice(0, 3) → first 3
slice(-4)   → last 4
```

56. Common Mistakes

Mistake 1 — `indexOf()` not found হলে `0` ভাবা



```
if (text.indexOf("x")) {
  console.log("Found");
}
```

সমস্যা হলো `indexOf()` `0` return করতে পারে, আর `0` falsy।

Better:

```
if (text.indexOf("x") !== -1) {  
    console.log("Found");  
}
```

অথবা আরও readable:

```
if (text.includes("x")) {  
    console.log("Found");  
}
```

57. Mistake 2 — `slice()` end index include করে ভাবা

```
const text = "Hello";  
  
console.log(text.slice(0, 2));
```

Output:

```
He
```

1 আসবে না।

কারণ:

```
start → included  
end   → excluded
```

58. Mistake 3 — `replace()` সব match replace করবে ভাবা

```
const text = "cat cat cat";  
  
console.log(text.replace("cat", "dog"));
```

Output:

```
dog cat cat
```

সব replace করতে:

```
text.replaceAll("cat", "dog");
```

59. Mistake 4 — `trim()` মাত্রের space remove করবে ভাবা

```
const text = "Hello World";  
  
console.log(text.trim());
```

Output:

```
Hello World
```

Middle space থাকবে।

60. Mistake 5 — `split()` original string modify করে

```
const text = "A-B-C";  
  
const result = text.split("-");
```

`text` এখনও:

```
A-B-C
```

আর `result`:

```
["A", "B", "C"]
```

61. Mistake 6 — `join()` String Method ভাবা

`join()` হলো **Array method**

```
const items = ["A", "B", "C"];

items.join("-");
```

আর:

```
const text = "A-B-C";

text.split("-");
```

এখানে:

```
split → String method
join  → Array method
```

62. String Methods Quick Table

Method	Input	Return
<code>indexOf()</code>	text	number
<code>includes()</code>	text	boolean
<code>slice()</code>	indexes	string
<code>split()</code>	separator	array
<code>join()</code>	separator	string
<code>trim()</code>	—	string
<code>replace()</code>	search + replacement	string
<code>replaceAll()</code>	search + replacement	string

63. Practice — Output Prediction

Question 1

```
const text = "JavaScript";

console.log(text.indexOf("Script"));
```

Question 2

```
const text = "Hello World";

console.log(text.includes("world"));
```

Question 3

```
const text = "JavaScript";

console.log(text.slice(4, 10));
```

Question 4

```
const text = "apple,banana,mango";

console.log(text.split(","));
```

Question 5

```
const items = ["JavaScript", "React", "Next.js"];

console.log(items.join(" | "));
```

Question 6

```
const text = "  Hello World  ";

console.log(text.trim());
```

Question 7

```
const text = "cat cat cat";

console.log(text.replace("cat", "dog"));
console.log(text.replaceAll("cat", "dog"));
```

Question 8

```
const name = "Shariar";
const age = 31;

console.log(`My name is ${name} and I am ${age}.`);
```

64. Coding Challenge 1 — Count Word Occurrence

একটি function তৈরি করো যেটা check করবে একটি word sentence-এর মধ্যে কতবার আছে।

Example:

```
countWord("I love JS. JS is powerful.", "JS");
```

Expected:

```
2
```

Hint:

```
split()
```

ব্যবহার করতে পারো।

65. Coding Challenge 2 — Clean Username

Input:

```
"  Shariar  "
```

Output:

```
"shariar"
```

Hint:

```
trim()  
toLowerCase()
```

66. Coding Challenge 3 — Convert Slug to Title

Input:

```
"learn-javascript-fast"
```

Output:

```
"Learn Javascript Fast"
```

Hint:

```
split()  
map()  
toUpperCase()  
slice()  
join()
```

67. Coding Challenge 4 — Mask Phone Number

Input:

```
01712345678
```

Output:

```
017****5678
```

Hint:

```
slice()
```

68. Coding Challenge 5 — Dynamic Greeting

Function:

```
createGreeting("Shariar", "Developer");
```

Expected:

```
Hello Shariar, you are a Developer.
```

এখানে template literal ব্যবহার করো।

69. Interview Questions

Beginner

1. What does `indexOf()` return?
2. What does `indexOf()` return when the value is not found?
3. What is the difference between `indexOf()` and `includes()`?
4. Is `includes()` case-sensitive?

5. How does `slice()` work?
6. Is the end index of `slice()` included?
7. What does `split()` return?
8. What does `join()` return?
9. What does `trim()` do?
10. What is the difference between `replace()` and `replaceAll()`?

Intermediate

11. Does `replace()` modify the original string?
12. Why does `indexOf()` create a bug when directly used in an `if` condition?
13. What is method chaining?
14. What are template literals?
15. What is string interpolation?
16. Can you execute expressions inside `${}`?
17. What are tagged templates?
18. What arguments does a tagged template function receive?
19. What is `String.raw()`?
20. Why are string methods generally returning new strings instead of modifying the original?

70. Quick Revision

Search

```
text.indexOf("hello");  
text.includes("hello");
```

```
indexOf → position  
includes → true / false
```

Extract

```
text.slice(0, 5);  
text.slice(-5);
```

String → Array

```
text.split(",");
```

Array → String

```
array.join(", ");
```

Clean

```
text.trim();  
text.trimStart();  
text.trimEnd();
```

Replace

```
text.replace("old", "new");  
text.replaceAll("old", "new");
```

Template Literal

```
`Hello ${name}`
```

Tagged Template

```
tag`Hello ${name}`;
```

Raw Template

```
String.raw`C:\Users\Shariar`;
```

71. Golden Rules

Rule 1: `indexOf()` → "কোথায় আছে?"

Rule 2: `includes()` → "আছে কি?"

Rule 3: `slice(start, end)` -এ `start` included, `end` excluded।

Rule 4: `split()` string-কে array বানায়।

Rule 5: `join()` array-কে string বানায়।

Rule 6: `trim()` শূন্য beginning এবং ending whitespace remove করে।

Rule 7: `replace()` সাধারণ string search-এর ক্ষেত্রে প্রথম match replace করে।

Rule 8: সব match replace করতে `replaceAll()` অথবা appropriate regex ব্যবহার করা যায়।

Rule 9: String methods original string mutate করে না; সাধারণত নতুন string return করে।

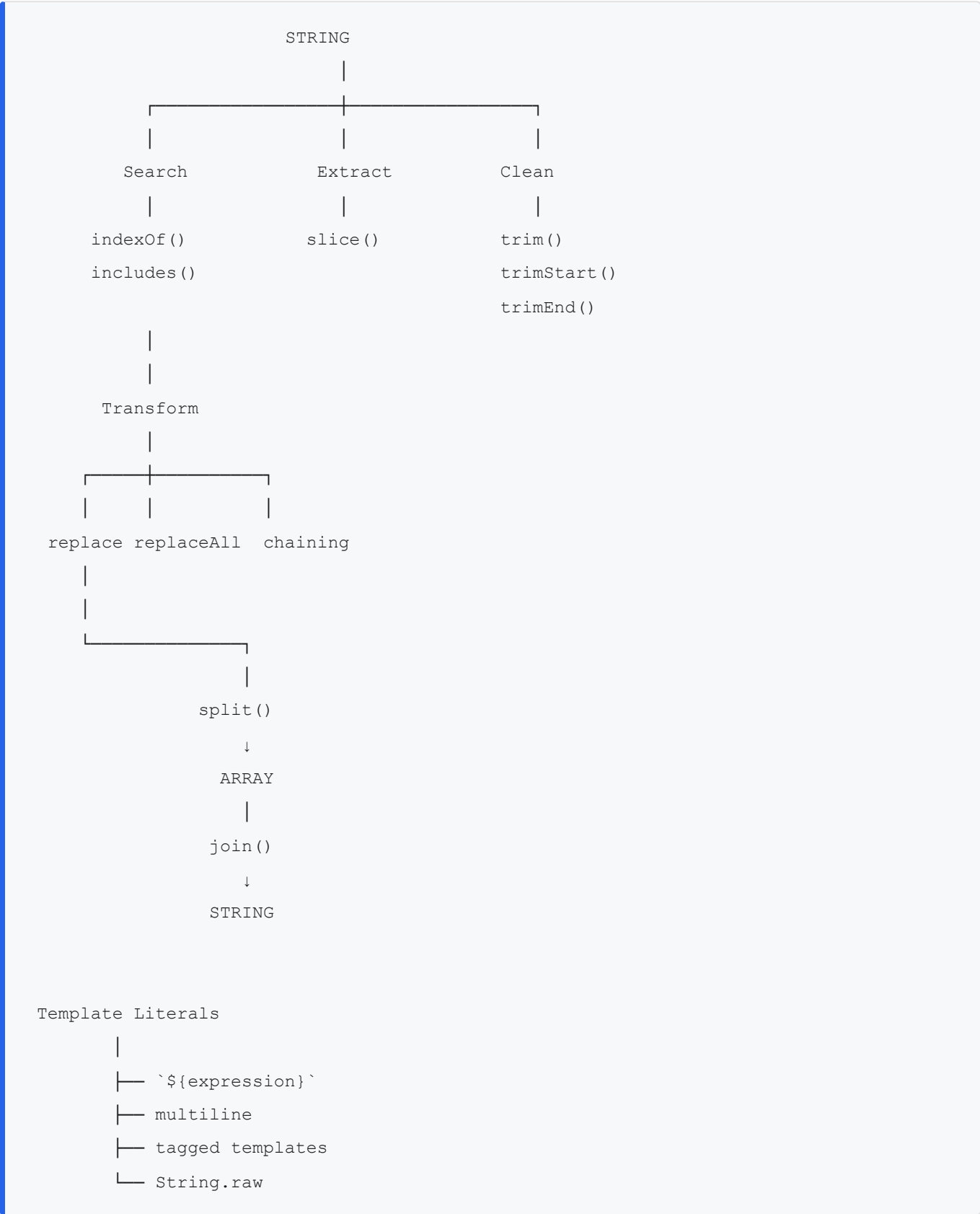
Rule 10: Template literals-এর জন্য backticks ব্যবহার হয়।

Rule 11: `${}` দিয়ে template literal-এর মধ্যে variable বা expression embed করা যায়।

Rule 12: Tagged templates হলো advanced template-literal feature যেখানে একটি function template-টিকে process করে।

Rule 13: `String.raw` raw template representation-এর জন্য useful।

Final Mental Model



Section 4 — Strings এখন complete!

Doc 4.1-এ তুমি String-এর **foundation, indexing, `length`, `at()`** এবং **immutability** শিখেছো। Doc 4.2-এ সেই foundation-এর উপর **search, extraction, transformation, splitting/joining** এবং **template literals** বসানো হয়েছে।